MLOps Assignment - Final Report

Heart Disease Prediction: End-to-End MLOps Pipeline Implementation

Course: MLOps (S2-25_AMLCSZG523)
Assignment: Assignment 1
Submitted By: Sannidhi Gokhale
Student ID: 2025CS05064
Date: May 2026

Deliverables

Part A: GitHub Repository

Repository URL: https://github.com/sanngokhale/heart-disease-mlops

The repository contains:

Item	Location
Source Code	`src/` (API, data processing, models)
Dockerfile	`Dockerfile` (multi-stage build)
Requirements	`requirements.txt`
Cleaned Dataset	`data/processed/heart_disease_preprocessed.csv`
Download Script	`src/data/download_data.py`
EDA Notebook	`notebooks/01_EDA.ipynb`
Training Script	`src/models/train_with_mlflow.py`
Unit Tests	`tests/` (27 tests)
CI/CD Workflow	`.github/workflows/ci.yml`
Kubernetes Manifests	`kubernetes/` (deployment, service, hpa)
Screenshots	`screenshots/`
Final Report	`docs/Final_Report.pdf`

Part B: Demo Video

Video link: https://app.vidcast.io/share/b881e02e-417f-4034-b4e5-a1e473485223

Google drive link (incase vidcast is not available):
https://drive.google.com/file/d/1_Mv4MZGSNSAMa0RxJThZRMDwohbXbjxM/view?usp=sharing

Duration: 3 minutes

Contents: End-to-end pipeline demonstration covering:

- Data preprocessing and model training with MLflow
- Model packaging (joblib serialization)
- Unit testing and Docker containerization
- Kubernetes deployment with auto-scaling
- API testing via Swagger UI
- Prometheus metrics monitoring

Part C: Deployed API Access

Local Testing Instructions:

```
# 1. Clone repository
git clone https://github.com/sanngokhale/heart-disease-mlops.git
cd heart-disease-mlops

# 2. Start Kubernetes deployment
kubectl apply -f kubernetes/

# 3. Port forward to access API
kubectl port-forward svc/heart-disease-api-service 8080:80 -n mlops-heart-
disease

# 4. Access endpoints
# Health: http://localhost:8080/health
# Swagger UI: http://localhost:8080/docs
# Predict: POST http://localhost:8080/predict
```

Table of Contents

---

1. Executive Summary

This report documents the implementation of a complete Machine Learning Operations (MLOps) pipeline for predicting heart disease risk using clinical patient data. The project demonstrates the application of modern MLOps best practices, from initial data exploration through production deployment and monitoring.

The pipeline processes the UCI Heart Disease dataset, trains multiple classification models, tracks experiments using MLflow, packages the best model into a Docker container, and deploys it to a Kubernetes cluster. The system includes automated CI/CD pipelines, comprehensive testing, and production-grade monitoring capabilities.

Key Achievements:

- Achieved 87% accuracy in heart disease prediction using Logistic Regression
- Implemented complete CI/CD pipeline with GitHub Actions
- Deployed scalable API on Kubernetes with auto-scaling (2-5 replicas)
- Integrated Prometheus metrics for production monitoring
- Created reproducible ML pipeline with MLflow experiment tracking

---

2. Introduction

2.1 Background

Heart disease remains one of the leading causes of mortality worldwide. Early detection and risk assessment are crucial for effective prevention and treatment. Machine learning models can assist healthcare professionals by providing data-driven risk predictions based on clinical indicators.

2.2 Project Objectives

The primary objective of this project is to implement a production-ready machine learning system that:

1. Analyzes clinical data to identify patterns associated with heart disease
2. Trains and evaluates multiple machine learning models
3. Tracks experiments for reproducibility
4. Packages the model for deployment
5. Deploys the model as a scalable API service
6. Monitors the system in production

2.3 Dataset Description

The UCI Heart Disease dataset (Cleveland) contains 303 patient records with 14 attributes:

Attribute	Description	Type
age	Patient age in years	Numeric
sex	Gender (1=male, 0=female)	Binary
cp	Chest pain type (0-3)	Categorical
trestbps	Resting blood pressure (mm Hg)	Numeric

Attribute	Description	Type
chol	Serum cholesterol (mg/dl)	Numeric
fbs	Fasting blood sugar > 120 mg/dl	Binary
restecg	Resting ECG results (0-2)	Categorical
thalach	Maximum heart rate achieved	Numeric
exang	Exercise-induced angina	Binary
oldpeak	ST depression induced by exercise	Numeric
slope	Slope of peak exercise ST segment	Categorical
ca	Number of major vessels colored by fluoroscopy	Categorical
thal	Thalassemia type	Categorical
target	Diagnosis of heart disease (0=no, 1=yes)	Binary

2.4 Technology Stack

- **Programming Language:** Python 3.11
- **ML Libraries:** scikit-learn, XGBoost, pandas, numpy
- **Experiment Tracking:** MLflow
- **API Framework:** FastAPI
- **Containerization:** Docker
- **Orchestration:** Kubernetes
- **CI/CD:** GitHub Actions
- **Monitoring:** Prometheus metrics

---

3. Part 1: Exploratory Data Analysis (EDA)

3.1 Data Loading and Initial Inspection

The dataset was loaded from the UCI Machine Learning Repository using a custom download script. Initial inspection revealed:

- **Total Records:** 303
- **Features:** 13 clinical attributes
- **Target Variable:** Binary classification (presence/absence of heart disease)
- **Missing Values:** Present in 'ca' and 'thal' columns (represented as '?')

3.2 Statistical Summary

Key statistics from the dataset:

Feature	Mean	Std Dev	Min	Max
age	54.4	9.0	29	77

Feature	Mean	Std Dev	Min	Max
trestbps	131.6	17.5	94	200
chol	246.3	51.8	126	564
thalach	149.6	22.9	71	202
oldpeak	1.04	1.16	0	6.2

3.3 Target Variable Distribution

The target variable shows a relatively balanced distribution:

- **No Disease (0):** 138 patients (45.5%)
- **Disease (1):** 165 patients (54.5%)

This balance is favorable for model training as it reduces the need for sampling techniques.

3.4 Key Findings from EDA

Age Distribution:

- Heart disease prevalence increases with age
- Highest risk group: 50-65 years
- Mean age of patients with disease: 56.6 years
- Mean age of patients without disease: 52.5 years

Gender Analysis:

- Male patients show higher heart disease rates (68% vs 32%)
- Dataset contains more male patients overall

Chest Pain Type (cp):

- Type 2 (non-anginal pain) shows strongest correlation with heart disease
- Asymptomatic patients (type 0) often have underlying disease

Maximum Heart Rate (thalach):

- Lower maximum heart rate correlates with heart disease
- Mean for disease: 139.1 bpm
- Mean for no disease: 158.5 bpm

ST Depression (oldpeak):

- Higher ST depression values indicate higher disease risk
- Strong predictor of heart disease

3.5 Correlation Analysis

A correlation matrix was generated to identify relationships between features:

- Strong positive correlation between age and cardiovascular risk indicators

- Negative correlation between maximum heart rate and disease presence
- Chest pain type shows significant correlation with target variable

3.6 Data Quality Assessment

Issues identified and addressed:

1. Missing values in 'ca' column: 4 records
2. Missing values in 'thal' column: 2 records
3. Data type inconsistencies in categorical columns
4. No duplicate records found

---

4. Part 2: Feature Engineering and Model Development

4.1 Data Preprocessing Pipeline

A comprehensive preprocessing pipeline was implemented:

Step 1: Missing Value Handling

- Strategy: Median imputation for numeric features
- Mode imputation for categorical features
- Rationale: Preserves data distribution while handling missing values

Step 2: Feature Type Separation

- Numeric features: age, trestbps, chol, thalach, oldpeak
- Categorical features: sex, cp, fbs, restecg, exang, slope, ca, thal

Step 3: Feature Transformation

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(drop='first', sparse_output=False),
         categorical_features)
    ]
)
```

4.2 Model Selection Rationale

Three models were selected for evaluation:

1. Logistic Regression

- Baseline model for binary classification
- Highly interpretable (important for medical applications)

- Fast training and inference
- Provides probability estimates

2. Random Forest

- Ensemble method handling non-linear relationships
- Robust to outliers and noise
- Provides feature importance rankings
- Handles mixed feature types well

3. Gradient Boosting (XGBoost)

- State-of-the-art for tabular data
- Handles imbalanced data well
- High predictive accuracy
- Built-in regularization

4.3 Training Process

The training process followed these steps:

1. **Data Split:** 80% training, 20% testing (stratified)
2. **Cross-Validation:** 5-fold CV for robust evaluation
3. **Metrics Tracked:** Accuracy, Precision, Recall, F1-Score, ROC-AUC

4.4 Model Comparison Results

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.87	0.86	0.91	0.88	0.92
Random Forest	0.85	0.84	0.88	0.86	0.91
Gradient Boosting	0.84	0.83	0.88	0.85	0.90

4.5 Best Model Selection

Selected Model: Logistic Regression

Justification:

1. Highest ROC-AUC score (0.92) indicating best discrimination ability
2. Highest F1-Score (0.88) balancing precision and recall
3. Best interpretability for medical domain
4. Fastest inference time for production deployment
5. Lower risk of overfitting on small dataset

4.6 Feature Importance Analysis

Top predictive features identified:

1. **cp (Chest Pain Type):** Strongest predictor

2. **thalach (Max Heart Rate):** Negative correlation with disease
3. **oldpeak (ST Depression):** Positive correlation with disease
4. **ca (Number of Vessels):** Important diagnostic indicator
5. **age:** Moderate positive correlation

---

5. Part 3: Experiment Tracking with MLflow

5.1 MLflow Setup

MLflow was configured for comprehensive experiment tracking:

```
import mlflow

mlflow.set_tracking_uri("./mlruns")
mlflow.set_experiment("heart-disease-classification")
```

5.2 Tracked Information

For each model run, the following was logged:

Parameters:

- Model type and hyperparameters
- Training/test split ratio
- Random state for reproducibility

Metrics:

- Accuracy, Precision, Recall, F1-Score
- ROC-AUC score
- Cross-validation mean and standard deviation

Artifacts:

- Trained model files (joblib format)
- Confusion matrix visualizations
- Feature importance plots
- Preprocessor pipeline

5.3 Experiment Runs Summary

Run ID	Model	ROC-AUC	CV Mean	Status
8b42207c	Logistic Regression	0.92	0.90	Best
c81912f8	Random Forest	0.91	0.88	-
f001f472	Gradient Boosting	0.90	0.87	-

5.4 MLflow UI Access

The MLflow tracking UI is accessible at:

```
mlflow ui --port 5001
# Access at http://127.0.0.1:5001
```

5.5 Benefits of Experiment Tracking

1. **Reproducibility:** Every run can be reproduced with logged parameters
2. **Comparison:** Easy comparison of model performance across runs
3. **Versioning:** Model artifacts are versioned automatically
4. **Collaboration:** Team members can view and compare experiments
5. **Audit Trail:** Complete history of experiments for compliance

---

6. Part 4: Model Packaging and Reproducibility

6.1 Model Artifacts

The following artifacts are saved for production use:

File	Description	Size
best_model.joblib	Trained Logistic Regression model	~1 KB
preprocessor.joblib	Feature transformation pipeline	~4.3 KB
model_info.json	Model metadata and training info	~500 B

6.2 Model Info Structure

```
{
    "model_name": "Logistic Regression",
    "model_type": "LogisticRegression",
    "training_date": "2026-05-03T16:51:02",
    "metrics": {
        "accuracy": 0.87,
        "precision": 0.86,
        "recall": 0.91,
        "f1_score": 0.88,
        "roc_auc": 0.92
    },
    "feature_columns": ["age", "sex", "cp", ...],
    "target_column": "target"
}
```

6.3 Reproducibility Measures

Environment Management:

- `requirements.txt` with pinned versions
- Python 3.11 specified
- All dependencies versioned

Code Version Control:

- Git repository with complete history
- Tagged releases for model versions

Data Versioning:

- Raw data stored in `data/raw/`
- Processed data in `data/processed/`
- Data download script for reproducibility

6.4 Loading and Using the Model

```
import joblib
import json

# Load artifacts
model = joblib.load("models/best_model.joblib")
preprocessor = joblib.load("models/preprocessor.joblib")

with open("models/model_info.json") as f:
    model_info = json.load(f)

# Make prediction
X_transformed = preprocessor.transform(input_data)
prediction = model.predict(X_transformed)
probability = model.predict_proba(X_transformed)[0][1]
```

---

7. Part 5: CI/CD Pipeline and Automated Testing

7.1 GitHub Actions Workflow

The CI/CD pipeline is defined in `.github/workflows/ci-cd.yml`:

```
name: MLOps CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
```

```
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Lint with flake8
        run: flake8 src/ tests/ --max-line-length=88

  test:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run tests
        run: pytest tests/ -v --cov=src

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Build Docker image
        run: docker build -t heart-disease-api .
```

7.2 Test Suite

Total Tests: 27 unit tests across 3 test files

Test File	Tests	Coverage
test_api.py	10	API endpoints
test_data_processing.py	9	Data pipeline
test_model.py	8	Model training

Test Categories:

1. **API Tests:**
   - Health endpoint validation
   - Prediction endpoint with valid/invalid input
   - Input validation (age, sex, ranges)
   - Error handling
2. **Data Processing Tests:**
   - Missing value handling
   - Data type validation
   - Preprocessing pipeline
3. **Model Tests:**
   - Feature engineering pipeline
   - Model loading and prediction

- Preprocessor functionality

7.3 Code Quality

Linting with flake8:

- Line length: 88 characters
- Import ordering checked
- Unused imports removed
- PEP 8 compliance

Test Results:

```
27 passed in 5.22s
```

---

8. Part 6: Model Containerization with Docker

8.1 Dockerfile

A multi-stage Dockerfile was created for optimized image size:

```dockerfile
# Stage 1: Builder
FROM python:3.11-slim as builder
WORKDIR /app
COPY requirements.txt .
RUN pip wheel --no-cache-dir -r requirements.txt

# Stage 2: Production
FROM python:3.11-slim
RUN groupadd -r appgroup && useradd -r -g appgroup appuser
WORKDIR /app

COPY --from=builder /app/wheels /wheels
RUN pip install --no-cache-dir /wheels/*

COPY src/ ./src/
COPY models/ ./models/

USER appuser
EXPOSE 8000
HEALTHCHECK --interval=30s CMD curl -f http://localhost:8000/health
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.2 Security Features

1. **Non-root user:** Application runs as `appuser`
2. **Minimal base image:** python:3.11-slim

3. **No unnecessary packages:** Only required dependencies
4. **Health checks:** Built-in container health monitoring

8.3 Docker Commands

```
# Build image
docker build -t heart-disease-api .

# Run container
docker run -d -p 8000:8000 --name heart-api heart-disease-api

# Test prediction
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"age": 63, "sex": 1, "cp": 3, ...}'
```

8.4 API Endpoints

Endpoint	Method	Description
/health	GET	Health check
/predict	POST	Make prediction
/model-info	GET	Model metadata
/metrics	GET	Prometheus metrics
/docs	GET	Swagger UI

---

9. Part 7: Production Deployment on Kubernetes

9.1 Kubernetes Architecture

The deployment uses Docker Desktop Kubernetes with the following resources:

Namespace: `mlops-heart-disease`

Deployment: 2 replicas with rolling update strategy

Service: LoadBalancer exposing port 80

HPA: Auto-scaling from 2 to 5 replicas based on CPU/memory

9.2 Kubernetes Manifests

namespace.yaml:

```
apiVersion: v1
kind: Namespace
```

```
metadata:
  name: mlops-heart-disease
```

deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: heart-disease-api
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: heart-disease-api
          image: heart-disease-api:latest
          ports:
            - containerPort: 8000
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
          livenessProbe:
            httpGet:
              path: /health
              port: 8000
```

service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: heart-disease-api-service
spec:
  type: LoadBalancer
  ports:
    - port: 80
      targetPort: 8000
```

9.3 Deployment Commands

```
# Apply manifests
kubectl apply -f kubernetes/namespace.yaml
kubectl apply -f kubernetes/deployment.yaml
```

```
kubectl apply -f kubernetes/service.yaml
kubectl apply -f kubernetes/hpa.yaml

# Verify deployment
kubectl get pods -n mlops-heart-disease
kubectl get svc -n mlops-heart-disease
```

9.4 Deployment Verification

```
NAME                                   READY   STATUS    RESTARTS   AGE
heart-disease-api-7685d94f68-5kjgl   1/1     Running   0          5m
heart-disease-api-7685d94f68-s2k6t   1/1     Running   0          5m
```

---

10. Part 8: Monitoring and Logging

10.1 Request Logging

FastAPI middleware logs all incoming requests:

```python
@app.middleware("http")
async def log_requests(request: Request, call_next):
    start_time = time.time()
    response = await call_next(request)
    process_time = time.time() - start_time

    logger.info(
        f"{request.method} {request.url.path} "
        f"- Status: {response.status_code} "
        f"- Duration: {process_time:.3f}s"
    )
    return response
```

Sample Log Output:

```
2026-05-03 17:27:53 - POST /predict - Status: 200 - Duration: 0.045s
2026-05-03 17:27:54 - GET /health - Status: 200 - Duration: 0.002s
```

10.2 Prometheus Metrics

Three custom metrics are exposed at `/metrics`:

```python
# Prediction counter by risk level
PREDICTIONS_TOTAL = Counter(
```

```
    "heart_disease_predictions_total",
    "Total predictions",
    ["risk_level"]
)

# Latency histogram
PREDICTION_LATENCY = Histogram(
    "heart_disease_prediction_latency_seconds",
    "Prediction latency"
)

# Request counter
REQUEST_COUNT = Counter(
    "heart_disease_api_requests_total",
    "Total requests",
    ["endpoint", "method", "status"]
)
```

10.3 Viewing Logs in Kubernetes

```
kubectl logs -f deployment/heart-disease-api -n mlops-heart-disease
```

10.4 Metrics Endpoint

```
curl http://localhost:8080/metrics | grep heart_disease

# Output:
heart_disease_predictions_total{risk_level="low_risk"} 5.0
heart_disease_predictions_total{risk_level="high_risk"} 3.0
heart_disease_prediction_latency_seconds_count 8.0
heart_disease_api_requests_total{endpoint="/predict",method="POST",status="
200"} 8.0
```

---

11. Conclusion

11.1 Summary of Achievements

This project successfully implemented a complete MLOps pipeline for heart disease prediction:

Component	Implementation	Status
EDA	Jupyter notebook with visualizations	✅ Complete
Feature Engineering	scikit-learn pipeline	✅ Complete
Model Training	3 models compared	✅ Complete

Component	Implementation	Status
Experiment Tracking	MLflow integration	✅ Complete
Model Packaging	Joblib artifacts	✅ Complete
CI/CD	GitHub Actions	✅ Complete
Containerization	Docker multi-stage	✅ Complete
Deployment	Kubernetes	✅ Complete
Monitoring	Prometheus + Logging	✅ Complete

11.2 Key Learnings

1. **MLOps Best Practices:** Importance of reproducibility, versioning, and automation
2. **Container Orchestration:** Kubernetes provides scalability and reliability
3. **Experiment Tracking:** MLflow enables systematic model comparison
4. **Production Readiness:** Health checks, logging, and monitoring are essential

11.3 Future Improvements

1. **Model Improvements:**
    - Hyperparameter tuning with GridSearchCV
    - Ensemble methods combining multiple models
    - Deep learning approaches for larger datasets
2. **Infrastructure:**
    - Helm charts for Kubernetes deployment
    - Grafana dashboards for visualization
    - Alerting on metric thresholds
3. **MLOps Enhancements:**
    - Model registry with MLflow
    - A/B testing for model versions
    - Data drift detection

---

12. References

1. UCI Machine Learning Repository - Heart Disease Dataset https://archive.ics.uci.edu/ml/datasets/heart+disease
2. MLflow Documentation https://mlflow.org/docs/latest/index.html
3. FastAPI Documentation https://fastapi.tiangolo.com/
4. Kubernetes Documentation https://kubernetes.io/docs/
5. Docker Documentation https://docs.docker.com/

6. scikit-learn Documentation https://scikit-learn.org/stable/

Appendix A: Repository Structure

```
heart-disease-mlops/
├── .github/workflows/ci-cd.yml
├── data/
│   ├── raw/heart_disease.csv
│   └── processed/heart_disease_preprocessed.csv
├── docs/
│   └── MLOps_Assignment_Report.md
├── kubernetes/
│   ├── namespace.yaml
│   ├── deployment.yaml
│   ├── service.yaml
│   └── hpa.yaml
├── models/
│   ├── best_model.joblib
│   ├── preprocessor.joblib
│   └── model_info.json
├── notebooks/
│   └── 01_EDA.ipynb
├── src/
│   ├── api/main.py
│   ├── data/
│   │   ├── download_data.py
│   │   └── preprocess.py
│   └── models/
│       ├── feature_engineering.py
│       └── train_with_mlflow.py
├── tests/
│   ├── test_api.py
│   ├── test_data_processing.py
│   └── test_model.py
├── Dockerfile
├── requirements.txt
└── README.md
```

Appendix B: API Request/Response Examples

Health Check:

```
GET /health

Response:
{
    "status": "healthy",
    "model_loaded": true,
```

```
    "preprocessor_loaded": true,
    "model_name": "Logistic Regression",
    "timestamp": "2026-05-03T17:27:37.560212"
}
```

Prediction Request:

```
POST /predict
{
    "age": 63, "sex": 1, "cp": 3, "trestbps": 145,
    "chol": 233, "fbs": 1, "restecg": 0, "thalach": 150,
    "exang": 0, "oldpeak": 2.3, "slope": 0, "ca": 0, "thal": 1
}

Response:
{
    "prediction": 0,
    "probability": 0.1272,
    "risk_level": "Low Risk",
    "confidence": 0.8728,
    "timestamp": "2026-05-03T17:27:53.131583",
    "model_version": "2026-05-03T16:51:02"
}
```

Repository Link: https://github.com/sanngokhale/heart-disease-mlops

---

End of Report